

CSS GRID

The Complete Developer Guide



Master two-dimensional layouts with CSS Grid —
from basics to advanced patterns

LAYOUT

CSS3

WEB DESIGN

FRONTEND

RESPONSIVE

Table of Contents

1. Introduction to CSS Grid	3
2. Grid Container Properties	4
3. Grid Item Properties	6
4. The fr Unit & repeat()	7
5. Grid Template Areas	8
6. Alignment & Justification	9
7. Auto Placement & Dense Packing	10
8. Responsive Grid Patterns	11
9. CSS Grid vs Flexbox	12
10. Common Layout Examples	13

1. Introduction to CSS Grid

CSS Grid Layout is a powerful two-dimensional layout system that allows you to design complex web layouts with rows **and** columns simultaneously. Unlike Flexbox, which works in one dimension at a time, Grid gives you full control over both axes, making it ideal for page-level layouts and complex component designs.

■ **Key Concept:** Grid works on a parent-child model. The *grid container* defines the grid structure using rows and columns, while *grid items* (direct children) are placed within that structure.

Enabling Grid Layout

```
/* Make any element a grid container */
.container {
  display: grid;
}

/* Inline grid (behaves like inline-block) */
.container {
  display: inline-grid;
}
```

Grid Terminology

Grid Container	The element with display: grid. All direct children become grid items.
Grid Item	Direct children of the grid container — placed into grid cells.
Grid Line	The dividing lines that form the grid structure (numbered 1, 2, 3...).
Grid Track	The space between two adjacent grid lines — a row or column.
Grid Cell	A single unit of the grid, the intersection of a row and column track.

Grid Area

A rectangular space bounded by four grid lines, can span multiple cells.

2. Grid Container Properties

grid-template-columns & grid-template-rows

Define the track sizes for columns and rows. Values can be fixed lengths, percentages, the **fr** unit, **auto**, or the **minmax()** function.

```
.container {
  display: grid;

  /* Fixed columns */
  grid-template-columns: 200px 200px 200px;

  /* Mixed units */
  grid-template-columns: 200px 1fr 2fr;

  /* Named lines */
  grid-template-columns: [sidebar-start] 250px [sidebar-end main-start] 1fr [main-end];

  /* Auto sizing */
  grid-template-rows: auto 1fr auto;
}
```

Visual: 3-column grid with mixed units



grid-gap / gap

Sets spacing between grid tracks. The modern shorthand **gap** replaced the older **grid-gap**.

```
.container {
  gap: 20px;           /* row & column gap */
  gap: 20px 10px;      /* row-gap column-gap */
  row-gap: 20px;
  column-gap: 10px;
}
```

grid-template-columns: repeat()

```
.container {  
  /* Repeat 3 equal columns */  
  grid-template-columns: repeat(3, 1fr);  
  
  /* Repeat a pattern */  
  grid-template-columns: repeat(2, 100px 200px);  
  /* Result: 100px 200px 100px 200px */  
  
  /* Auto-fill: as many as fit */  
  grid-template-columns: repeat(auto-fill, minmax(200px, 1fr));  
  
  /* Auto-fit: collapse empty tracks */  
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));  
}
```

Container Property Reference

grid-template-columns	<code>track-size... repeat()</code>	Defines column track sizes
grid-template-rows	<code>track-size... repeat()</code>	Defines row track sizes
grid-template-areas	<code>'area names...'</code>	Defines named template areas
grid-template	<code>shorthand</code>	Shorthand for rows, columns, areas
column-gap		Space between column tracks
row-gap		Space between row tracks
gap		Shorthand for row-gap column-gap
justify-items	<code>start end center stretch</code>	Aligns items along row axis
align-items	<code>start end center stretch</code>	Aligns items along column axis
place-items		Shorthand for align/justify-items

justify-content	start end center stretch space-*	Aligns grid within container (row axis)
align-content	start end center stretch space-*	Aligns grid within container (col axis)
grid-auto-columns	track-size	Size of implicitly created columns
grid-auto-rows	track-size	Size of implicitly created rows
grid-auto-flow	row column dense	Controls auto-placement algorithm

3. Grid Item Properties

Placing Items with grid-column & grid-row

Grid items can be explicitly placed using line numbers, named lines, or the **span** keyword to stretch across multiple tracks.

```
.item {
  /* Shorthand: start-line / end-line */
  grid-column: 1 / 3;      /* spans columns 1-2 */
  grid-row: 1 / 2;        /* row 1 only */

  /* Using span */
  grid-column: 2 / span 2; /* start at 2, span 2 cols */
  grid-row: span 3;        /* auto-place, span 3 rows */

  /* Using -1 (last line) */
  grid-column: 1 / -1;     /* full width */
}
```

Visual: Item spanning columns 1–3

Item A (spans 3 cols)	Item B
Item C	Item D

Item Property Reference

grid-column-start	span	Where item starts on column axis
grid-column-end	span	Where item ends on column axis
grid-column	start / end	Shorthand for column-start/end
grid-row-start	span	Where item starts on row axis
grid-row-end	span	Where item ends on row axis

grid-row	<code>start / end</code>	Shorthand for row-start/end
grid-area	<code>name r/c/r/c</code>	Named area or row/col shorthand
justify-self	<code>start end center stretch</code>	Aligns item within cell (row axis)
align-self	<code>start end center stretch</code>	Aligns item within cell (col axis)
place-self		Shorthand for align/justify-self

4. The fr Unit & repeat()

The **fr** (fraction) unit represents a fraction of the available space in the grid container. It distributes remaining space *after* fixed and auto-sized tracks are accounted for.

■ **How fr works:** If you have `1fr 2fr 1fr`, the total is 4 parts. The first and last columns get 1/4 each, the middle gets 2/4 (half) of available space.

```
.container {  
  /* Three equal columns */  
  grid-template-columns: 1fr 1fr 1fr;  
  
  /* Sidebar + main content */  
  grid-template-columns: 250px 1fr;  
  
  /* Two sidebars + main */  
  grid-template-columns: 200px 3fr 200px;  
  
  /* minmax() with fr */  
  grid-template-columns: repeat(3, minmax(100px, 1fr));  
}
```

Visual: 1fr 2fr 1fr distribution



minmax() Function

The **minmax(min, max)** function sets a size range for a track, allowing tracks to be flexible within defined constraints.

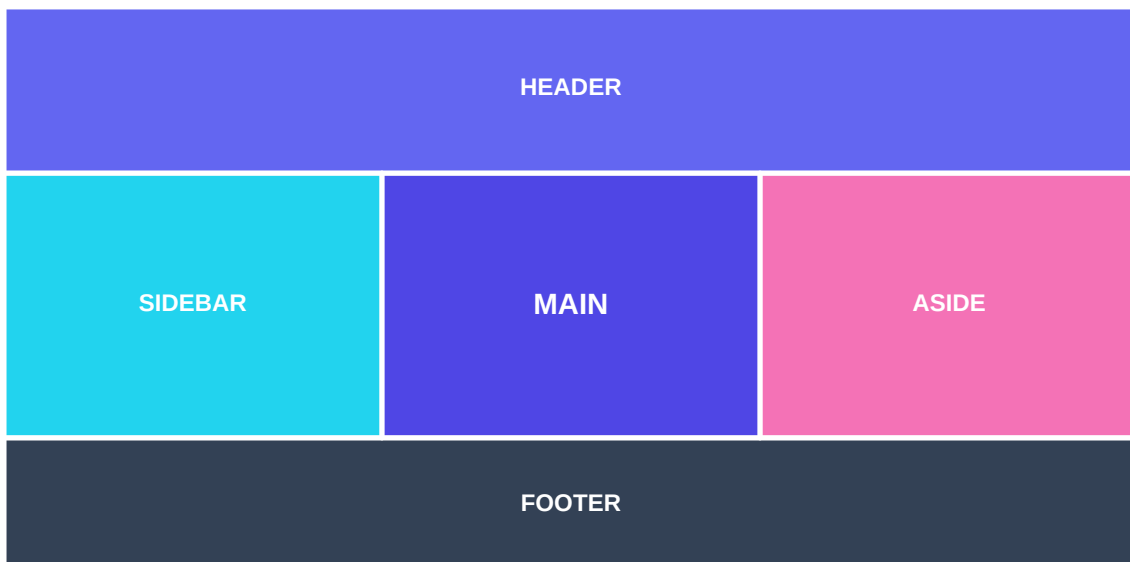
```
.container {  
  /* Column: at least 150px, at most 1fr */  
  grid-template-columns: repeat(3, minmax(150px, 1fr));  
  
  /* Row: at least 50px, auto grow */  
  grid-template-rows: minmax(50px, auto);  
  
  /* Common responsive pattern */  
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));  
}
```

5. Grid Template Areas

Named template areas allow you to build layouts using readable ASCII-art syntax. Each string in **grid-template-areas** represents a row, with area names defining which cells belong to each section.

```
.container {  
  display: grid;  
  grid-template-columns: 220px 1fr 220px;  
  grid-template-rows: 70px 1fr 60px;  
  grid-template-areas:  
    "header header header"  
    "sidebar main  aside"  
    "footer footer footer";  
  gap: 16px;  
  min-height: 100vh;  
}  
  
.header { grid-area: header; }  
.sidebar { grid-area: sidebar; }  
.main { grid-area: main; }  
.aside { grid-area: aside; }  
.footer { grid-area: footer; }
```

Visual: Classic page layout using grid-template-areas



■ **Tip:** Use a period . in grid-template-areas to leave a cell empty: "sidebar . main"

6. Alignment & Justification

CSS Grid provides fine-grained control over alignment along both axes. Properties prefixed with **justify-** operate on the inline (row) axis, while **align-** properties operate on the block (column) axis.

Property	Target	Description	Values
justify-items	Container	Aligns all items horizontally within their cells	start end center stretch
align-items	Container	Aligns all items vertically within their cells	start end center stretch
justify-content	Container	Distributes grid columns within the container	start end center stretch space-between space-around space-evenly
align-content	Container	Distributes grid rows within the container	start end center stretch space-between space-around space-evenly
justify-self	Item	Overrides horizontal alignment for a single item	start end center stretch
align-self	Item	Overrides vertical alignment for a single item	start end center stretch

```
/* Center all items in their cells */
.container {
  justify-items: center;
  align-items: center;
  /* shorthand: place-items: center; */
}

/* Center the entire grid within the container */
.container {
  justify-content: center;
  align-content: center;
  /* shorthand: place-content: center; */
}

/* Override for a single item */
.special-item {
  justify-self: end;
  align-self: start;
  /* shorthand: place-self: start end; */
}
```

7. Auto Placement & Dense Packing

When grid items are not explicitly placed, the auto-placement algorithm positions them automatically. The **grid-auto-flow** property controls how this algorithm works.

```
.container {  
  grid-template-columns: repeat(3, 1fr);  
  
  /* Default: fill row by row */  
  grid-auto-flow: row;  
  
  /* Fill column by column */  
  grid-auto-flow: column;  
  
  /* Dense: fill holes left by spanning items */  
  grid-auto-flow: row dense;  
  
  /* Size implicitly created rows */  
  grid-auto-rows: minmax(100px, auto);  
}
```

■ **Dense packing:** When some items span multiple tracks, gaps can appear. Adding **dense** to **grid-auto-flow** tells the browser to fill those holes with smaller items — note this may change visual order.

8. Responsive Grid Patterns

Auto-fill vs Auto-fit

Both keywords work with **repeat()** to create responsive grids without media queries. The difference is how they handle empty tracks.

```
/* auto-fill: creates empty tracks to fill the row */
.auto-fill {
  grid-template-columns: repeat(auto-fill, minmax(200px, 1fr));
}

/* auto-fit: collapses empty tracks (items stretch more) */
.auto-fit {
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
}
```

Common Responsive Patterns

RAM Pattern (Repeat, Auto, Minmax)

```
grid-template-columns: repeat(auto-fit, minmax(min(100%, 300px), 1fr));
```

✓ *Truly responsive — never overflows on small screens*

Fixed Sidebar

```
grid-template-columns: clamp(200px, 25%, 350px) 1fr;
```

✓ *Sidebar shrinks but stays within bounds*

Holy Grail

```
grid-template:
  'h h h' 60px
  's m a' 1fr
  'f f f' 50px / 200px 1fr 200px;
```

✓ *Full page layout in a few lines*

Card Grid

```
grid-template-columns: repeat(auto-fill, minmax(280px, 1fr));  
gap: 24px;
```

✓ *Responsive card layout without media queries*

9. CSS Grid vs Flexbox

Grid and Flexbox are complementary tools — not competitors. Understanding when to use each leads to cleaner, more maintainable code.

Feature	CSS Grid	Flexbox
Dimensions	2D (rows AND columns)	1D (row OR column)
Layout Control	Container-driven	Content-driven
Item Sizing	Defined by tracks	Defined by items
Best For	Page/complex layouts	Component/nav layouts
Overlap	Easy (z-index)	Complex
Browser Support	Excellent (95%+)	Excellent (98%+)
Alignment	Both axes independently	One axis primary
Implicit Tracks	Yes (grid-auto-*)	No

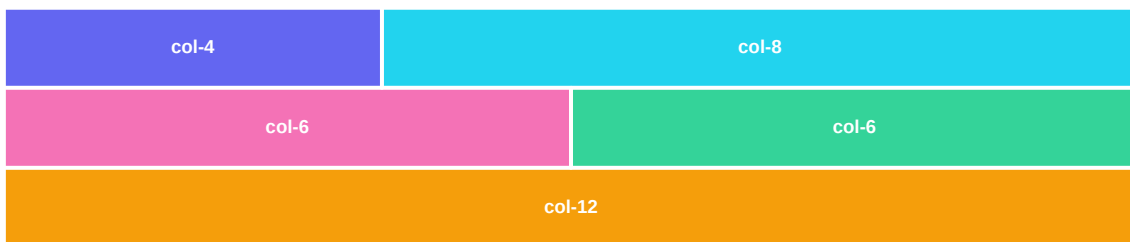
■ **Rule of thumb:** Use Grid for the overall page structure and large layout regions. Use Flexbox for navigation bars, toolbars, card content, form rows, and anywhere items need to wrap naturally based on their content size.

10. Common Layout Examples

12-Column Grid System

```
.grid {  
  display: grid;  
  grid-template-columns: repeat(12, 1fr);  
  gap: 16px;  
}  
.col-4 { grid-column: span 4; } /* 1/3 width */  
.col-6 { grid-column: span 6; } /* 1/2 width */  
.col-8 { grid-column: span 8; } /* 2/3 width */  
.col-12 { grid-column: span 12; } /* full width */
```

Visual: 12-column grid — 4+8, then 6+6, then 12



Masonry-Style Layout

```
.masonry {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
  grid-auto-rows: 80px;  
  gap: 12px;  
}  
.tall { grid-row: span 2; }  
.short { grid-row: span 1; }  
.wide { grid-column: span 2; }
```

Full-Page App Shell

```
body {  
  display: grid;  
  grid-template-rows: 60px 1fr 40px;  
  grid-template-columns: 240px 1fr;  
  grid-template-areas:  
    "nav    header"  
    "nav    content"  
    "nav    footer";  
  height: 100vh;  
}  
  
nav    { grid-area: nav; overflow-y: auto; }  
header { grid-area: header; }  
.content { grid-area: content; overflow-y: auto; }  
footer { grid-area: footer; }
```

■ Quick Reference Cheat Sheet

Container: display:grid | grid-template-columns | grid-template-rows |
grid-template-areas | gap | justify-items | align-items | grid-auto-flow

Items: grid-column | grid-row | grid-area | justify-self | align-self

Units & Functions: fr | repeat() | minmax() | auto-fill | auto-fit | span